instance_exec()

instance_eval has a slightly more flexible twin brother named instance_exec that allows you to pass arguments to the block. This feature is useful in a few rare cases, such as the one in this artfully complicated example:

blocks/instance_exec.rb

```
class C
  def initialize
    @x = 1
  end
end

class D
  def twisted_method
    @y = 2
    C.new.instance_eval { "@x: #{@x}, @y: #{@y}" }
  end
end

D.new.twisted_method  # => "@x: 1, @y: "
```

You might assume that the block in D#twisted_method can access both the @x instance variable from C and the @y instance variable from D in the same *Flat Scope* (83). However, instance variables depend on self, so when instance_eval switches self to the receiver, all the instance variables in the caller fall out of scope. The code inside the block interprets @y as an instance variable of C that hasn't been initialized, and as such is nil (and prints out as an empty string).

To merge @x and @y in the same scope, you can use instance_exec to pass @y's value to the block:

```
class D
  def twisted_method
    @y = 2
➤   C.new.instance_exec(@y) {|y| "@x: #{@x}, @y: #{y}" }
  end
end

D.new.twisted_method  # => "@x: 1, @y: 2"
```

Another acceptable reason to break encapsulation is arguably testing. Here's an example.

The Padrino Example

The Padrino web framework defines a Logger class that manages all the logging that a web application must deal with. The Logger stores its own configuration into instance variables. For example, @log_static is true if the application must log access to static files.